**TASK 9 – Implement and Test the Calculator**

**9.1 Complete Calculator**

The following program uses Decimal for monetary values and validates the inputs.

```
from decimal import Decimal, ROUND_HALF_UP, InvalidOperation


BASE_RATE = Decimal("30.00")

STANDARD_FAT = Decimal("3.5")

STANDARD_SNF = Decimal("8.5")

FAT_STEP_VALUE = Decimal("0.40")

SNF_STEP_VALUE = Decimal("0.30")

STEP_SIZE = Decimal("0.1")


CENT = Decimal("0.01")




def money(value):

    """Round a monetary value to the nearest paisa."""

    return value.quantize(CENT, rounding=ROUND_HALF_UP)




def rate_per_litre(fat, snf):

    """Calculate the rate per litre."""

    fat_steps = int(

        ((fat - STANDARD_FAT) / STEP_SIZE).to_integral_value(
```

```
            rounding=ROUND_HALF_UP
        )
    )

    snf_steps = int(
        ((snf - STANDARD_SNF) / STEP_SIZE).to_integral_value(
            rounding=ROUND_HALF_UP
        )
    )

    rate = (
        BASE_RATE
        + Decimal(fat_steps) * FAT_STEP_VALUE
        + Decimal(snf_steps) * SNF_STEP_VALUE
    )

    return money(rate)



def payout(litres, fat, snf):
    """Calculate the amount payable for one entry."""
    if litres <= 0:
        raise ValueError("Litres must be greater than zero.")
```

```
    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Fat must be between 2.0 and 7.0.")

    if snf < 0:
        raise ValueError("SNF cannot be negative.")

    return money(litres * rate_per_litre(fat, snf))


def ten_day_total(payments):
    """Calculate the sum of individual farmer payments."""
    return sum((money(p) for p in payments), Decimal("0.00"))


def run_tests():
    """Test normal, boundary and invalid entries."""
    tests = [
        ("Standard", "10", "3.5", "8.5", "300.00"),
        ("Above standard", "8.5", "4.2", "8.7", "283.90"),
        ("Below standard", "12", "3.1", "8.3", "333.60"),
        ("Minimum fat", "10", "2.0", "8.5", "240.00"),
        ("Maximum fat", "10", "7.0", "8.5", "440.00")
    ]
```

```
for name, litres, fat, snf, expected in tests:
    actual = payout(
        Decimal(litres),
        Decimal(fat),
        Decimal(snf)
    )

    assert actual == Decimal(expected), name
    print(name, "Expected:", expected, "Actual:", actual)

invalid_tests = [
    ("Negative quantity", "-1", "3.5", "8.5"),
    ("Fat below minimum", "10", "1.9", "8.5"),
    ("Fat above maximum", "10", "7.1", "8.5")
]

for name, litres, fat, snf in invalid_tests:
    try:
        payout(
            Decimal(litres),
            Decimal(fat),
            Decimal(snf)
        )
    except ValueError:
```

```
            print(name, "PASS: invalid entry rejected")
        else:
            raise AssertionError(name)


if __name__ == "__main__":
    run_tests()
```

**9.2 Expected Test Results**

| Test | Input | Expected Result |
|---|---|---|
| Standard entry | 10 L, 3.5% fat, 8.5% SNF | ₹300.00 |
| Above standard | 8.5 L, 4.2% fat, 8.7% SNF | ₹283.90 |
| Below standard | 12 L, 3.1% fat, 8.3% SNF | ₹333.60 |
| Minimum fat | 10 L, 2.0% fat, 8.5% SNF | ₹240.00 |
| Maximum fat | 10 L, 7.0% fat, 8.5% SNF | ₹440.00 |
| Negative quantity | -1 L | Rejected |
| Fat below minimum | 1.9% | Rejected |
| Fat above maximum | 7.1% | Rejected |

**9.3 Five-Entry Total Calculated by Hand**

For this example, assume the following five entries:

| Entry | Litres | Fat | SNF | Rate | Payment |
|---|---|---|---|---|---|
| 1 | 10 | 3.5 | 8.5 | ₹30.00 | ₹300.00 |
| 2 | 8.5 | 4.2 | 8.7 | ₹33.40 | ₹283.90 |
| 3 | 12 | 3.1 | 8.3 | ₹27.80 | ₹333.60 |
| 4 | 5 | 3.6 | 8.5 | ₹30.40 | ₹152.00 |

| | | | | | |
|---|---|---|---|---|---|
| 5 | 6 | 3.5 | 8.6 | ₹30.30 | ₹181.80 |
| **Total** | | | | | **₹1,251.30** |

### 9.4 Alternative Concept: Manual Testing vs Automated Testing

Manual testing involves entering every input and checking the output individually.

Automated testing uses assertions:

```
assert payout(
    Decimal("10"),
    Decimal("3.5"),
    Decimal("8.5")
) == Decimal("300.00")
```

If a calculation changes, the same test can be executed again automatically.

For n test cases, running the assertions takes O(n) time, excluding the cost of individual calculations. Manual testing also requires O(n) effort to check n cases and is more vulnerable to human error.

**Conclusion:** Automated testing is recommended because it makes regression testing faster, repeatable and more reliable.

**TASK 10 – Compare Expectations with Execution**

**10.1 Float vs Decimal Totals**

The assignment requires comparing the results for 300, 6,000 and 150,000 payments.

The following program measures the time taken and compares the totals.

```
from decimal import Decimal
from time import perf_counter

PAYMENT = "33.40"


def calculate_float(n):
    """Calculate total using float."""
    total = 0.0

    for _ in range(n):
        total += float(PAYMENT)

    return total


def calculate_decimal(n):
    """Calculate total using Decimal."""
    total = Decimal("0.00")
    amount = Decimal(PAYMENT)
```

```
    for _ in range(n):
        total += amount

    return total


for n in (300, 6000, 150000):
    start = perf_counter()
    float_total = calculate_float(n)
    float_time = perf_counter() - start

    start = perf_counter()
    decimal_total = calculate_decimal(n)
    decimal_time = perf_counter() - start

    print("Payments:", n)
    print("Float total:", format(float_total, ".2f"))
    print("Decimal total:", decimal_total)
    print("Difference:", abs(
        Decimal(str(float_total)) - decimal_total
    ))
    print("Float time:", float_time)
    print("Decimal time:", decimal_time)
```

```
print()
```

**Expected Mathematical Totals**

For a payment of ₹33.40:

| Number of Payments | Expected Total |
|---|---|
| 300 | ₹10,020.00 |
| 6,000 | ₹200,400.00 |
| 150,000 | ₹5,010,000.00 |

The actual floating-point differences and execution times must be obtained by running the program on your computer. Do not invent measurement results for the assignment report.

**10.2 Why Differences Occur**

Binary floating-point numbers cannot represent all decimal fractions exactly. When many values are added, small errors may accumulate.

Decimal arithmetic can represent decimal amounts exactly when the inputs are created from decimal strings and the precision is sufficient.

**10.3 Rounding Each Payment vs Rounding the Total**

Consider a rate of ₹33.335 per litre and a quantity of 1 litre.

If each payment is rounded to two decimal places using half-up rounding, the payment becomes ₹33.34.

For 100 such payments:

- Rounding each payment first gives ₹3,334.00.
- Summing the unrounded values gives ₹3,333.50.

These totals differ because the first method applies rounding to every payment.

For this assignment, the recommended rule is:

1. Calculate the rate.
2. Round the rate to two decimal places.

3. Calculate the payment.
4. Round each payment to two decimal places.
5. Add the rounded payments to obtain the bank total.

This rule ensures that the total equals the sum of the amounts actually payable to individual farmers.

The co-operative must confirm the official rounding policy before production deployment.

**10.4 Alternative Concept: Precomputed Rate Table**

If many farmers have the same fat and SNF readings, their rates can be calculated once and stored in a dictionary.

```
rate_table = {
    ("3.5", "8.5"): Decimal("30.00"),
    ("4.2", "8.7"): Decimal("33.40"),
    ("3.1", "8.3"): Decimal("27.80")
}


key = ("4.2", "8.7")
rate = rate_table[key]


print(rate)
```

Output:

33.40

**Complexity Comparison**

| **Method** | **Rate calculation** | **Storage** |
|---|---|---|
| Calculate every time | O(1) per entry | O(1) extra space |
| Dictionary lookup | O(1) average lookup | O(k), where k is the number of distinct pairs |

Both methods process n entries in O(n) expected time.

However, the dictionary approach is only beneficial if readings repeat frequently and can be represented using consistent keys. With two-decimal analyser readings, many distinct fat/SNF combinations may exist.

A rate table should also be associated with the correct seasonal rate card to avoid returning outdated rates.

**TASK 11 – Recommend and Defend the Final Design**

**11.1 Recommended Design**

The recommended design consists of:

1. Named constants or a rate-card configuration.
2. Decimal arithmetic for monetary calculations.
3. Separate functions for rate calculation, payment and total.
4. A module containing reusable calculation functions.
5. Input validation before calculations.
6. Automated tests using assertions.
7. A running total or stored transaction records, depending on reporting requirements.

**11.2 Recommended Types**

| **Data** | **Recommended Type** | **Reason** |
|---|---|---|
| Farmer ID | str | Identifies each farmer |
| Shift | str | Identifies the collection period |
| Milk quantity | Decimal | Supports decimal quantities |
| Fat and SNF | Decimal | Preserves decimal readings |
| Rates and money | Decimal | Supports accurate decimal arithmetic |
| Step counts | int | Represents whole increments |
| Entries | list of records | Organizes transactions |

**11.3 Rate Revision**

The original rate card should be defined in one place:

BASE_RATE = Decimal("30.00")

FAT_STEP_VALUE = Decimal("0.40")

SNF_STEP_VALUE = Decimal("0.30")

When the co-operative revises a rate, the relevant values can be updated centrally.

For seasonal rates, a rate-card function or configuration file is preferable to repeatedly changing expressions throughout the code.

**11.4 Limitations**

- The program assumes the input data is correct after validation.
- It does not directly connect to a milk analyser or bank.
- It requires a defined policy for rounding and analyser measurement precision.
- It needs persistent storage and transaction records for production use.
- It does not yet include authentication, audit logging or backup procedures.

**11.5 Suitability for District-Union Scale**

The district union may process 150,000 farmer entries.

Since each entry can be calculated independently, the calculation requires O(n) time.

If all entries are stored, storage is O(n). If only a running total is maintained, the additional calculation storage can be O(1), excluding the input and output systems.

For 150,000 entries, a well-written Python implementation is a reasonable starting point. However, the system should be benchmarked with realistic data before deployment.

**11.6 Alternative Concept: External Rate-Card File**

Instead of storing rates in the Python module, they can be read from a JSON file.

Example rate_card.json:

```
{
   "base_rate": "30.00",
   "standard_fat": "3.5",
```

```
    "standard_snf": "8.5",
    "fat_step_value": "0.40",
    "snf_step_value": "0.30"
}
```

Python can load the file:

```
import json
from decimal import Decimal

with open("rate_card.json", "r") as file:
    data = json.load(file)

BASE_RATE = Decimal(data["base_rate"])
FAT_STEP_VALUE = Decimal(data["fat_step_value"])
SNF_STEP_VALUE = Decimal(data["snf_step_value"])
```

This allows authorized staff to revise the rate card without editing the program's source code.

Reading t configuration values requires O(t) time and O(t) space.

Although file loading introduces a small startup cost, it makes rate management easier and reduces the need to redistribute modified source code for every revision.

**Final Recommendation:** Retain the modular structure, modify the rate calculation to use Decimal inputs and explicit rounding rules, and introduce configurable seasonal rate cards.

**TASK 12 – DAY-4 DESIGN CHALLENGE: Re-evaluate and Adapt the Design**

**A. Impact Analysis**

**12.A.1 Changes in Requirements**

The new requirements introduce three major changes:

1. The automatic milk analyser provides fat and SNF readings to two decimal places.
2. From April to June, the base rate becomes ₹32.00 per litre.
3. Each fat step is worth ₹0.45 instead of ₹0.40.
4. Farmers supplying more than 500 litres in ten days receive a bonus of ₹0.50 per litre.

The original design must be modified because the rates are no longer fixed for every season, and the bonus depends on the farmer's total quantity over ten days.

**12.A.2 Affected Components**

| **Component** | **Required Change** |
|---|---|
| Constants | Add seasonal rate-card values |
| Fat step calculation | Handle two-decimal readings |
| SNF step calculation | Handle two-decimal readings |
| Rate function | Accept a seasonal rate card |
| Payment function | Calculate the payment using the selected rate card |
| Bonus function | Add a bonus for qualifying farmers |
| Ten-day total function | Track each farmer's total quantity and payment |

**12.A.3 Two-Decimal Readings**

The original step formula is:

$$\text{Fat Steps} = \text{round}\left(\frac{\text{Fat} - 3.5}{0.1}\right)$$

For an analyser reading of 4.23% fat:

$$\frac{4.23 - 3.5}{0.1} = 7.3$$

The reading does not correspond to a whole 0.1% step.

Therefore, the system must specify how fractional steps are handled. In this solution, the number of steps is rounded to the nearest whole step using **round-half-up**.

For 4.23% fat, the number of fat steps is 7.

For 4.26% fat, the number of fat steps is 8.

This rule must be approved by the co-operative because different rounding policies may produce different payments.

**B. Evaluation of the Original Design**

The original design already contains useful components:

- Named constants.
- Separate calculation functions.
- A reusable module.
- Input validation.
- Automated testing.

These features remain appropriate.

However, the original fixed constants must be replaced or supplemented by seasonal rate cards. The rate function must also handle two-decimal readings using a consistent step-rounding policy.

**Worked Example 1: Original Season**

Assume:

- Litres = 10
- Fat = 4.20%

- SNF = 8.70%
- Base rate = ₹30.00
- Fat step = ₹0.40
- SNF step = ₹0.30

Fat steps:

$$\frac{4.20 - 3.50}{0.10} = 7$$

SNF steps:

$$\frac{8.70 - 8.50}{0.10} = 2$$

Rate:

$$30 + (7 \times 0.40) + (2 \times 0.30)$$
$$= 30 + 2.80 + 0.60$$
$$= ₹33.40$$

Payment:

$$10 \times 33.40 = ₹334.00$$

**Worked Example 2: April–June Season**

Assume:

- Litres = 10
- Fat = 4.20%
- SNF = 8.70%
- Base rate = ₹32.00
- Fat step = ₹0.45

- SNF step remains ₹0.30 because no new SNF adjustment was specified.

Rate:

$$32 + (7 \times 0.45) + (2 \times 0.30)$$
$$= 32 + 3.15 + 0.60$$
$$= ₹35.75$$

Payment:

$$10 \times 35.75 = ₹357.50$$

The seasonal rate card changes the amount payable while preserving the original modular design.

**Evaluation:** The original design should be **modified rather than completely redesigned**. Its functions and module remain useful, but the rate card, step calculation and bonus logic need to be updated.

**C. Adaptation or Redesign**

**Marks: 2**

The revised program will use:

1. A rate card for the original season.
2. A separate rate card for April–June.
3. Decimal arithmetic for quantities, percentages and money.
4. Explicit round-half-up rules for step counts and payments.
5. A function that calculates the rate.
6. A function that calculates an individual payment.
7. A function that calculates the ten-day bonus.

### 12.C.1 Complete Revised Python Program

Save the following code as seasonal_milkrate.py.

```
from decimal import Decimal, ROUND_HALF_UP


CENT = Decimal("0.01")

STEP_SIZE = Decimal("0.1")


STANDARD_FAT = Decimal("3.5")

STANDARD_SNF = Decimal("8.5")


# Original rate card

ORIGINAL_RATE_CARD = {

    "base": Decimal("30.00"),

    "fat_step": Decimal("0.40"),

    "snf_step": Decimal("0.30")

}


# April-June rate card

SUMMER_RATE_CARD = {

    "base": Decimal("32.00"),

    "fat_step": Decimal("0.45"),

    "snf_step": Decimal("0.30")

}
```

```
BONUS_THRESHOLD = Decimal("500")
BONUS_PER_LITRE = Decimal("0.50")



def round_money(value):
    """Round a monetary amount to the nearest paisa."""
    return value.quantize(CENT, rounding=ROUND_HALF_UP)



def whole_steps(value):
    """Round a step count to the nearest whole step."""
    return int(
        value.to_integral_value(rounding=ROUND_HALF_UP)
    )



def select_rate_card(season):
    """Return the rate card for the specified season."""
    if season == "april_june":
        return SUMMER_RATE_CARD

    if season == "original":
        return ORIGINAL_RATE_CARD
```

```
    raise ValueError("Unknown season.")



def rate_per_litre(fat, snf, season):
    """Calculate the rate using the selected seasonal rate card."""
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))

    rate_card = select_rate_card(season)

    fat_steps = whole_steps(
        (fat - STANDARD_FAT) / STEP_SIZE
    )

    snf_steps = whole_steps(
        (snf - STANDARD_SNF) / STEP_SIZE
    )

    rate = (
        rate_card["base"]
        + Decimal(fat_steps) * rate_card["fat_step"]
        + Decimal(snf_steps) * rate_card["snf_step"]
    )
```

```
    return round_money(rate)



def payout(litres, fat, snf, season):
    """Calculate the payment for one milk entry."""
    litres = Decimal(str(litres))
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))

    if litres <= 0:
        raise ValueError("Litres must be greater than zero.")

    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Fat must be between 2.0 and 7.0.")

    if snf < 0:
        raise ValueError("SNF cannot be negative.")

    rate = rate_per_litre(fat, snf, season)

    return round_money(litres * rate)



def ten_day_bonus(total_litres):
```

```
    """Calculate the ten-day bonus for a farmer."""
    total_litres = Decimal(str(total_litres))

    if total_litres > BONUS_THRESHOLD:
        return round_money(total_litres * BONUS_PER_LITRE)

    return Decimal("0.00")



def final_ten_day_payment(total_milk_payment, total_litres):
    """Add the qualifying bonus to the milk payment total."""
    total_milk_payment = Decimal(str(total_milk_payment))

    bonus = ten_day_bonus(total_litres)

    return round_money(total_milk_payment + bonus)



if __name__ == "__main__":
    print("Original season")

    original_rate = rate_per_litre(
        Decimal("4.20"),
        Decimal("8.70"),
```

```
    "original"
)

original_payment = payout(
    Decimal("10"),
    Decimal("4.20"),
    Decimal("8.70"),
    "original"
)

print("Rate:", original_rate)
print("Payment:", original_payment)

print("\nApril-June season")

summer_rate = rate_per_litre(
    Decimal("4.20"),
    Decimal("8.70"),
    "april_june"
)

summer_payment = payout(
    Decimal("10"),
    Decimal("4.20"),
```

```
        Decimal("8.70"),
        "april_june"
    )

    print("Rate:", summer_rate)
    print("Payment:", summer_payment)

    print("\nBonus tests")

    print("500 litres:", ten_day_bonus(Decimal("500")))
    print("501 litres:", ten_day_bonus(Decimal("501")))

    print(
        "Final payment for 501 litres with milk payment "
        "of Rs 18000:",
        final_ten_day_payment(
            Decimal("18000"),
            Decimal("501")
        )
    )
```

**12.C.2 Expected Output**

```
Original season
Rate: 33.40
Payment: 334.00
```

April-June season

Rate: 35.75

Payment: 357.50

Bonus tests

500 litres: 0.00

501 litres: 250.50

Final payment for 501 litres with milk payment of Rs 18000: 18250.50

**Explanation of the Bonus**

The bonus applies only when the farmer supplies **more than 500 litres** in ten days.

For exactly 500 litres:

$$\text{Bonus} = ₹0.00$$

For 501 litres:

$$\text{Bonus} = 501 \times 0.50$$
$$= ₹250.50$$

The final ten-day payment is the sum of the milk payment and the bonus.

**Assumption:** The bonus is applied to the total ten-day quantity, as stated in the changed requirement. The example uses the milk payment supplied to the final-payment function; in a production system, this must be the sum of the farmer's individual payments.

**D. Evidence-Based Defence**

The modified design is preferable because it meets the new requirements without discarding the working parts of the original solution.

### 12.D.1 Test Results

| Test | Season | Litres | Fat | SNF | Rate | Milk Payment |
|---|---|---|---|---|---|---|
| 1 | Original | 10 | 3.50 | 8.50 | ₹30.00 | ₹300.00 |
| 2 | Original | 10 | 4.20 | 8.70 | ₹33.40 | ₹334.00 |
| 3 | April–June | 10 | 3.50 | 8.50 | ₹32.00 | ₹320.00 |
| 4 | April–June | 10 | 4.20 | 8.70 | ₹35.75 | ₹357.50 |

### 12.D.2 Bonus Tests

| Ten-Day Quantity | Bonus Rate | Bonus Amount |
|---|---|---|
| 499 litres | ₹0.50/L | ₹0.00 |
| 500 litres | ₹0.50/L | ₹0.00 |
| 501 litres | ₹0.50/L | ₹250.50 |
| 600 litres | ₹0.50/L | ₹300.00 |

### 12.D.3 Advantages

1. Seasonal rates can be selected without rewriting the calculation functions.
2. Decimal arithmetic supports accurate monetary calculations.
3. The step-rounding rule is explicit and testable.
4. The bonus is handled separately from the ordinary milk rate.
5. The same program can support different seasons and larger numbers of farmers.
6. Automated tests can detect incorrect rate revisions.

### 12.D.4 Limitations and Trade-Offs

- The co-operative must confirm the rounding rule for two-decimal analyser readings.
- The system needs a clearly defined SNF validation range.

- The seasonal rate card currently changes only the base rate and fat-step value; the SNF-step value remains unchanged because the question does not specify a new value.
- The bonus rule must be applied to each farmer's total quantity across the complete ten-day period, not separately to each delivery.
- A production system should save transaction records and maintain an audit trail.